

INSOFT S.A.S

ENTREGA DE PROYECTO

Documentación Técnica: Componente TConversacion (ISW)

Equipo de Desarrollo ISW

Noviembre, 2025

1. Introducción

1.1. Propósito

`TConversacion` es un componente Svelte de alto nivel, disponible en el paquete `@jeff-ayorta/svelteprueba`, diseñado para encapsular y abstraer la totalidad de la lógica de una interfaz de conversación en tiempo real.

Su objetivo principal es proporcionar una solución "plug-and-play" que minimice drásticamente el tiempo de desarrollo. Al consumir este componente, el desarrollador se abstrae de la gestión manual de:

- Conexiones WebSocket y reconexiones (vía Azure Web PubSub).
- Gestión de estado de mensajes (enviando, enviado, editado, leído).
- Paginación y carga de historial (scroll infinito).
- Renderizado condicional de tipos de mensajes (citas, reacciones, eliminados).
- Lógica de UI para citar, editar y reaccionar a mensajes.

1.2. Arquitectura de Consumo

Para integrar `TConversacion`, es fundamental comprender la interacción de las tres entidades del ecosistema:

1. ISW-Chat (Frontend - El Consumidor):

- Es el proyecto (Astro/Svelte) que importa y renderiza `TConversacion`.
- Es responsable de instanciar y configurar el `TMensajeClient` con las credenciales y la URL del backend.

2. ISS (Backend API - El Servidor):

- Una API de Azure Functions que expone la lógica de negocio.
- **API de Persistencia (REST):** Provee endpoints CRUD (Crear, Leer, Actualizar, Borrar) para la entidad `Mensaje`.
- **Broker de Conexión (WebSocket):** Expone un endpoint (ej. `/api/pubsub`) que genera una URL de conexión temporal a Azure Web PubSub. `TConversacion` utiliza esta URL para establecer la conexión en tiempo real.

3. ISP-Chat (Lógica/Tipos - El Paquete):

- Un paquete NPM (`@jeff-aporta/ispchat`) que actúa como librería compartida para mantener la consistencia contractual entre frontend y backend.
- **Contratos de Datos (Tipos):** Define las interfaces TypeScript (ej. `TMensaje` , `TConversacion` , `TReaccion`) que ambos, ISW e ISS, utilizan.
- **Cliente API:** Exporta la clase `TMensajeClient` , un *wrapper* que encapsula la lógica `fetch` para interactuar con la API de ISS.

IMAGEN PENDIENTE (Código MMD)

Descripción: Diagrama de arquitectura de componentes.

```

1      ``mermaid
2      graph TD
3      subgraph "<span style='white-space: nowrap;'>ISP (Paquete
4          Compartido)</span>"
5          Types(Tipos: TMensaje, etc.)
6          Client(TMensajeClientInterface)
7      end
8
9      subgraph "<span style='white-space: nowrap;'>ISW (Frontend)
10         </span>"
11         App(Consumidor App) -- Renderiza --> TCon(TConversacion)
12         App -- Implementa --> Client
13     end
14
15     subgraph "<span style='white-space: nowrap;'>ISS (Backend
16         API)</span>"
17         API[API REST (Endpoint /mensaje)]
18         PubSubURL(Endpoint /pubsub)
19         PubSubService[Azure Web PubSub]
20     end
21
22     TCon -- Llama a 'obtenerMensajes()' --> Client
23     Client -- 1. HTTP Request (REST) --> API
24
25     TCon -- Llama a 'obtenerUrlPubSub()' --> Client
26     Client -- 2. HTTP Request --> PubSubURL
27     PubSubURL -- 3. Genera URL --> Client
28     Client -- 4. Devuelve URL --> TCon
29     TCon -- 5. Conexión WebSocket --> PubSubService
30
31     ISW -- Importa Tipos --> Types
32     ISS -- Importa Tipos --> Types
33     ``

```

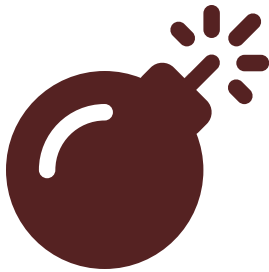
1.3. El Patrón de Diseño: Inversión de Dependencias

El diseño de `TConversacion` se basa fundamentalmente en el principio de Inversión de Dependencias (la "D" de SOLID) para lograr un **Acoplamiento Débil**.

La Solución (Acoplamiento Débil): La arquitectura invierte esta dependencia. En lugar de que la Vista dependa de la Lógica, ambos dependen de una abstracción (la Interfaz).

1. **La Vista (`TConversacion`):** Solo conoce el "Contrato" (la interfaz). Su única responsabilidad es renderizar la UI y llamar a los métodos de ese contrato (ej. `props.controlador.fnEnviarMensaje()`).
2. **El Contrato (`TMensajeClientInterface`):** Define el "QUÉ" (qué métodos deben existir), pero no el "CÓMO" (cómo se implementan).
3. **El Controlador (Su implementación):** Es la lógica concreta que el consumidor crea e inyecta en el componente. Aquí es donde vive el `fetch`, la lógica de autenticación y el manejo de URLs.

Este patrón (conocido como **Inyección de Dependencias**) es la base de esta arquitectura, permitiendo que el componente sea testeable, mantenible y agnóstico a la fuente de datos.



Syntax error in text
mermaid version 11.12.0

Código Mermaid (Diagrama de Dependencias):

```

1      ---
2      config:
3      theme: default
4      ---
5      flowchart TB
6
7      subgraph subGraph1["<span style='white-space: nowrap;'>Dependencias</span>"].
8          D("TMensajeClientInterface")
9          C("TConversacion")
10         E("Implementación del Cliente")
11     end
12     C --> D
  
```

13

E --> D

14

2. Paso 1: El Contrato del Cliente

(TMensajeClientInterface)

Como se describe en el patrón de Inversión de Dependencias, al componente `TConversacion` no le debe importar **cómo** se implementa la lógica de backend (es decir, **cómo** se hacen las llamadas API, **cómo** se gestiona la autenticación, o qué librería se usa).

Al componente solo le debe interesar que el objeto `mensajeClient` que se le inyecta (como prop) cumpla con un "Contrato" formal. Este contrato es la interfaz `TMensajeClientInterface`, la cual se alinea con una metodología de "Diseño Basado en Interfaz" (Interface-First Design).

Cualquier clase que implemente la siguiente interfaz (exportada desde `@jeff-ayponta/ispchat/client`) será un cliente válido:

2.1. El Contrato: TMensajeClientInterface

```
1 // Interfaz para gestionar recursos de mensajes
2 export interface TMensajeClientInterface {
3   // elimina un mensaje del servidor
4   fnEliminarMensaje(mensaje: TMensaje): Promise<boolean>;
5
6   // envia un mensaje, cita o reaccion al servidor
7   fnEnviarMensaje(mensaje: TMensaje): Promise<boolean>;
8
9   // edita el contenido de un mensaje existente
10  fnEditarMensaje(mensaje: TMensaje): Promise<boolean>;
11
12  // obtiene mensajes del servidor
13  obtenerMensajes(args: {
14    sentido: TSentidoTiempo;
15    mensaje?: TMensaje;
16    limite?: number
17  }): Promise<TMensaje[]>;
18
19  // obtiene informacion de usuario para mostrar en mensaje
20  obtenerInformacionUsuario(args: {
21    mensaje: TMensaje;
22    mensajes: TMensaje[]
23  }): TInformacionUsuario;
24
25  // obtiene la URL de conexión para Web PubSub
26  obtenerUrlPubSub(args: {
27    iusuario: string;
28    hub: string
```

```
29   }): Promise<string>;  
30 }  
31
```

El consumidor del componente es responsable de crear (o importar) una clase que implemente esta interfaz y de gestionar la lógica de conexión (REST, GraphQL, etc.) y autenticación (Tokens) dentro de dicha clase.

3. Paso 2: Implementación Mínima

El siguiente código es la implementación mínima funcional del componente.

```
1 <script lang="ts">
2   import { TConversacion, TDTipoConversacion } from "@jeff-aporta/svelteprueba";
3   import { TMensajeClient } from "@jeff-aporta/ispchat/client";
4
5   // --- 1. Configuración del Cliente (Paso 1) ---
6   // El consumidor debe proveer su propia implementación de
7   // TMensajeClientInterface.
8   // A modo de ejemplo, asumimos que 'miCliente' es una instancia válida.
9   const miCliente: TMensajeClientInterface = new MiImplementacionDeCliente();
10
11  // --- 2. Parámetros Requeridos ---
12  // ID del usuario que está utilizando la instancia del chat
13  export const iusuarioActual: string = "USUARIO_LOGUEADO"; // Ej: "PRENDON"
14  // ID de la sala o grupo de chat
15  export const idConversacion: string = "SALA_DE_PRUEBA"; // Ej: "TEST"
16 </script>
17
18 <!-- 3. Renderizado del Componente -->
19 <!-- Es CRÍTICO definir una altura en el componente o en su contenedor padre -->
20 <div style="height: 500px; border: 1px solid #ccc;">
21   <TConversacion
22     mensajeClient={miCliente}
23     {iusuarioActual}
24     {idConversacion}
25     ttdconversacion={TDTipoConversacion.chat}
26   />
27 </div>
```

Consideración Crítica: Definición de Altura

Observe el `<div>` que envuelve a `TConversacion` con `style="height: 500px;"`. Esto es **fundamental**. El componente de chat está diseñado para llenar el 100% de la altura de su contenedor padre (`height: 100%` por defecto). El área de historial de mensajes necesita una altura **definida** para saber cuándo debe volverse "scrollable".

Si no defines una altura en el contenedor padre, el chat intentará crecer verticalmente de forma infinita a medida que se cargan mensajes, en lugar de crear una barra de scroll interna.

IMAGEN PENDIENTE

Descripción: Captura de pantalla del componente `TConversacion` funcionando con la implementación básica.

Formato: PNG.

Contenido: Mostrar el chat sin estilos personalizados, con la cabecera y la entrada de mensajes que vienen por defecto.

4. Paso 3: API de Propiedades (Props)

A continuación, se detallan todas las propiedades (props) expuestas al consumidor, analizadas desde `Conversacion.svelte` y `Historial.svelte`.

4.1. Props Requeridas (Core)

Estas propiedades son mandatorias para la inicialización y funcionalidad del componente.

Prop	Tipo	Descripción
<code>mensajeClient</code>	<code>TMensajeClientInterface</code>	(Mandatoria) La instancia configurada del cliente. Es el motor para toda la persistencia y la obtención de la URL de PubSub.
<code>iusuarioActual</code>	<code>string</code>	(Mandatoria) El identificador único del usuario actual. Se utiliza internamente para: 1. Determinar la alineación de los mensajes (enviados vs. recibidos). 2. Registrar lecturas (no se auto-registra la lectura). 3. Habilitar permisos de edición y eliminación (solo en mensajes propios). 4. Identificar la autoría de reacciones.
<code>idConversacion</code>	<code>string</code>	(Mandatoria) El identificador único de la sala de chat. Se utiliza internamente para: 1. Solicitar el historial de mensajes correcto a la API. 2. Unirse al grupo de Web PubSub correspondiente para recibir eventos en tiempo real.
<code>ttdconversacion</code>	<code>TTDTipoConversacion</code>	(Mandatoria) Un <i>enum</i> que define el modo de renderizado. Valor: <code>TTDTipoConversacion.chat</code> . Actúa como un selector de modo. Aunque actualmente solo <code>chat</code> está implementado, esta prop permite la extensibilidad futura (ej. <code>TTDTipoConversacion.foro</code>).

4.2. Props Opcionales (Control de UX)

Estas propiedades ajustan el comportamiento y la experiencia de usuario. Se definen en `TConversacion` y se propagan internamente a `TChat` y `THistorial`.

Prop	Tipo	Default	Descripción
<code>enviarConEnter</code>	<code>boolean</code>	<code>true</code>	Define el comportamiento de la tecla "Enter" en el campo de entrada. <code>true</code> : "Enter" envía el mensaje. "Shift+Enter" crea una nueva línea. (Comportamiento de WhatsApp/Telegram). <code>false</code> : "Enter" crea una nueva línea. "Ctrl+Enter" (o "Cmd+Enter") envía el mensaje. (Comportamiento de Slack/Discord).
<code>mostrarAvatarUsuario</code>	<code>boolean</code>	<code>true</code>	Controla la visibilidad del componente <code>TAvatar</code> junto a los mensajes. <code>true</code> : Renderiza el <code>TAvatar</code> . <code>false</code> : Oculta el <code>TAvatar</code> para una interfaz más minimalista.
<code>avatarAncho</code>	<code>string number</code>	<code>18</code>	Define el ancho (y alto) del <code>TAvatar</code> . Acepta un número (ej. <code>32</code>) que se interpreta como <code>px</code> , o un string (ej. <code>"2.5rem"</code>).
<code>mostrarNombreSobrePrimerMensaje</code>	<code>boolean</code>	<code>true</code>	Controla la agrupación visual de mensajes. <code>true</code> : Agrupa mensajes consecutivos del mismo usuario, mostrando el nombre del autor solo sobre el primer mensaje del bloque. <code>false</code> : Muestra el nombre del autor en <i>cada</i> mensaje.
<code>mostrarCabecera</code>	<code>boolean</code>	<code>true</code>	Controla la visibilidad de la <code>TTarjeta</code> superior que envuelve el <code>slot="cabecera"</code> .

Prop	Tipo	Default	Descripción
			<code>true</code> : Renderiza el espacio de la cabecera. <code>false</code> : Oculta la cabecera, y el historial de mensajes ocupa el espacio superior (layout de 2 filas en lugar de 3).

4.3. Props Opcionales (Control de Estilo)

Estas propiedades permiten la personalización visual del componente.

Prop	Tipo	Default	Descripción
<code>style</code>	<code>string Record<...></code>	<code>""</code>	Aplica estilos CSS en línea al componente raíz <code>TTarjeta</code> (el contenedor principal). Es el método recomendado para definir la altura (ej. <code>style={{ height: "80vh" }}</code>).
<code>styleHistorial</code>	<code>string Record<...></code>	<code>""</code>	Aplica estilos CSS en línea directamente al contenedor del historial (<code>THistorial</code>). Es ideal para <code>backgroundImage</code> (ej. <code>styleHistorial={{ backgroundImage: 'url(...)' }}</code>).
<code>historialThemeName</code>	<code>string undefined</code>	<code>undefined</code>	Asigna el atributo <code>data-theme-name</code> al componente <code>THistorial</code> . Útil para aplicar temas CSS personalizados (ej. "dark", "light") si están definidos en el proyecto consumidor.

5. Paso 4: API de Slots (Personalización)

`TConversacion` expone dos slots principales (`cabecera` y `entradaMensaje`) que permiten al consumidor inyectar su propio contenido Svelte/HTML, reemplazando las secciones por defecto.

5.1. Slot: `cabecera`

Reemplaza el contenido de la barra superior del chat. Es ideal para mostrar el nombre de la sala, los participantes, o contenido multimedia relevante.

Ejemplo (Basado en `_index.svelte`):

```
1 <script lang="ts">
2   import { TDiv } from "@jeff-aporta/svelteprueba";
3   // ... resto del script
4 </script>
5
6 <TConversacion
7   {...propsRequeridas}
8   style={{ height: "500px" }}
9 >
10  <!--
11    Contenido personalizado para la cabecera: un video de YouTube.
12    (Basado en el archivo _index.svelte proporcionado)
13  -->
14  <TDiv slot="cabecera" class="flex align-center gap-10px">
15    <iframe
16      width="560"
17      height="315"
18      src="https://www.youtube.com/embed/s7C3m_NHaGg?si=HWXVU6TomFInnkC1"
19      title="YouTube video player"
20      frameborder="0"
21      allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope;
picture-in-picture; web-share"
22      referrerpolicy="strict-origin-when-cross-origin"
23      allowfullscreen
24    ></iframe>
25  </TDiv>
26 </TConversacion>
27
```

IMAGEN PENDIENTE

Descripción: Captura de pantalla del chat con un slot de cabecera personalizado.

Formato: PNG.

Contenido: Mostrar el chat con el ejemplo de cabecera (el video de YouTube incrustado).

5.2. Slot: entradaMensaje

Reemplaza el área de entrada de texto inferior.

Caso 1: Ejemplo Simple (Reemplazo de Input)

Si el objetivo es solo reemplazar el `input` por defecto (ej. por un `TCampoTexto` o un `input` HTML estándar) pero **conservar el formulario, la lógica de envío, y el botón de enviar por defecto** del componente `TConversacion`, simplemente pase el componente de input al slot. `TConversacion` (`THistorial`) está diseñado para buscar un `input` o `textarea` dentro de este slot.

```
1 <script lang="ts">
2   import { TCampoTexto } from "@jeff-aporta/svelteprueba";
3   // ... resto del script
4 </script>
5
6 <TConversacion {...propsRequeridas} style={{ height: "500px" }}>
7   <!--
8     Opción 1: Un <input> HTML estándar. (Como en _index.svelte)
9     TConversacion lo encontrará y lo usará.
10  -->
11   <input slot="entradaMensaje" placeholder="Escribe un mensaje (HTML)" />
12
13   <!--
14     Opción 2: Usar un componente TCampoTexto.
15     (Basado en el código comentado en _index.svelte)
16  -->
17   <!--
18   <TCampoTexto
19     slot="entradaMensaje"
20     variante="texto"
21     placeholder="Escribe un mensaje (TCampoTexto)"
22     borderWidth={1}
23     borderColor="rgba(128,128,128,0.3)"
24   />
25   -->
26 </TConversacion>
27
```

Caso 2: Ejemplo Avanzado (Reemplazo Total del Formulario)

Advertencia: Si se inyecta contenido que *no* es un simple `input` (ej. botones propios, `divs` envoltorios), `TConversacion` asume que el consumidor tomará control total del formulario.

Esto significa que el consumidor es ahora responsable de:

1. Renderizar sus propios botones (`Adjuntar` , `Enviar`).
2. Capturar el evento de envío (clic en botón o "Enter").
3. Instanciar el objeto `TMensaje` (o `TCita`).
4. Llamar a `mensajeClient.fnEnviarMensaje(nuevoMensaje)` manualmente.
5. Limpiar el input y el estado de la cita/edición.

```
1 <script lang="ts">
2   import { TMensaje } from "@jeff-aporta/ispchat/client"; // Importar el tipo
3   // ... resto del script
4
5   let miMensaje = "";
6
7   async function enviarMensaje() {
8     if (miMensaje.trim() === "") return;
9
10    // 1. Crear el objeto TMensaje (de ispchat)
11    const nuevoMensaje = new TMensaje({
12      idconversacion: idConversacion,
13      iusuario: iusuarioActual,
14      contenido: miMensaje,
15      // ... otros campos como 'icita' si estuvieras citando
16    });
17
18    try {
19      // 2. Usar el cliente para enviar el mensaje a la API (ISS)
20      await mensajeClient.fnEnviarMensaje(nuevoMensaje);
21
22      // 3. Limpiar el input
23      miMensaje = "";
24    } catch (error) {
25      console.error("Error al enviar mensaje:", error);
26      // Implementar manejo de errores (ej. Toast)
27    }
28  }
29 </script>
30
31 <TConversacion {...propsRequeridas} style={{ height: "500px" }}>
32   <!-- Contenido personalizado que REEMPLAZA el <form> -->
33   <div slot="entradaMensaje" class="custom-input-area">
34     <button class="attach-button">+</button>
35     <input
36       type="text"
```

```

37     placeholder="Escribe tu mensaje..."
38     class="custom-input"
39     bind:value={miMensaje}
40     on:keydown={(e) => e.key === 'Enter' && enviarMensaje()}
41   />
42   <button class="send-button" on:click={enviarMensaje}>
43     📩 Enviar
44   </button>
45 </div>
46 </TConversacion>
47
48 <style>
49   .custom-input-area {
50     display: flex;
51     padding: 10px;
52     border-top: 1px solid #ccc;
53     gap: 8px;
54   }
55   .custom-input {
56     flex-grow: 1;
57     border: 1px solid #ddd;
58     border-radius: 4px;
59     padding: 8px;
60   }
61   .attach-button, .send-button {
62     background: #f0f0f0;
63     border: 1px solid #ccc;
64     border-radius: 4px;
65     cursor: pointer;
66     padding: 0 12px;
67   }
68 </style>
69

```

IMAGEN PENDIENTE

Descripción: Captura de pantalla del chat con un slot de entrada avanzado.

Formato: PNG.

Contenido: Mostrar el chat con los botones personalizados  y , reemplazando la entrada por defecto.

6. Paso 5: Ejemplo Completo (Referencia del Consumidor)

Este código es una versión limpia del archivo `_index.svelte` proporcionado, integrada en la estructura de esta documentación técnica.

6.1. Componente `ChatPage.svelte` (Ejemplo Completo)

```
1 <script lang="ts">
2   // Importar estilos locales del consumidor
3   import "../_index.scss";
4
5   import { TConversacion, TDTipoConversacion, TDiv } from "@jeff-
aporta/svelteprueba";
6   import { TMensajeClient } from "@jeff-aporta/ispchat/client";
7
8   // --- Configuración (Paso 1) ---
9   // Asumimos que 'miCliente' es una instancia válida que implementa
TMensajeClientInterface
10  const miCliente: TMensajeClientInterface = new MiImplementacionDeCliente();
11  // (Aquí iría la lógica real para instanciar y configurar el cliente)
12
13
14  // --- Parámetros desde URL (Basado en _index.svelte) ---
15  // (Esta función 'obtenerParametro' es un ejemplo de cómo el consumidor
16  // puede obtener los datos. No es parte del componente TConversacion.)
17  function obtenerParametro(nombre: string, valorPorDefecto: string): string {
18    if (typeof window === "undefined") return valorPorDefecto;
19    const urlActual = new URL(window.location.href);
20    let valor = (urlActual.searchParams.get(nombre) ?? "").trim();
21    if (!valor) {
22      valor = valorPorDefecto;
23      urlActual.searchParams.set(nombre, valor);
24      window.history.replaceState(null, "", urlActual.toString());
25    }
26    return valor;
27  }
28  export const iusuarioActual: string = obtenerParametro("iusuario", "VISITANTE");
29  export const idConversacion: string = obtenerParametro("grupo", "GENERAL"); //
Nota: el nombre del parámetro "grupo" es flexible.
30 </script>
31
32 <!--
33   Este 'main' actúa como el contenedor de la página,
34   dando un fondo y centrando el componente de chat.
```

```

35 -->
36 <main class="chat-container">
37   <TConversacion
38     mensajeClient={miCliente}
39     {iusuarioActual}
40     {idConversacion}
41     ttdconversacion={TTDTipoConversacion.chat}
42
43     <!-- Props de Estilo (Mandatorio definir altura) -->
44     style={{
45       height: "80vh",
46       maxWidth: "800px",
47       margin: "0 auto",
48       border: "1px solid #e0e0e0",
49       borderRadius: "8px",
50       boxShadow: "0 4px 12px rgba(0,0,0,0.1)",
51       overflow: "hidden" // Asegura que las esquinas redondeadas corten el
contenido
52     }}
53     styleHistorial={{
54       // Fondo sutil para el área de mensajes (como en tu ejemplo)
55       backgroundImage:
`url("https://www.transparenttextures.com/patterns/inspiration-geometry.png")`,
56     }}
57
58     <!-- Props de UX -->
59     enviarConEnter={true}
60     mostrarAvatarUsuario={true}
61     avatarAncho={32}
62   >
63     <!-- Slot de Cabecera Personalizado (de _index.svelte) -->
64     <TDiv slot="cabecera" class="flex align-center gap-10px">
65       <iframe
66         width="560"
67         height="315"
68         src="https://www.youtube.com/embed/s7C3m_NHaGg?si=HWXVU6TomFIInnkCl"
69         title="YouTube video player"
70         frameborder="0"
71         allow="accelerometer; autoplay; clipboard-write; encrypted-media;
gyroscope; picture-in-picture; web-share"
72         referrerpolicy="strict-origin-when-cross-origin"
73         allowfullscreen
74       ></iframe>
75     </TDiv>
76
77     <!-- Slot de Entrada Personalizado (Caso 1 - Simple) -->
78     <input slot="entradaMensaje" placeholder="Escribe un mensaje" />
79
80   </TConversacion>
81 </main>

```

```
82
83 <style>
84   /* Estilos globales para la página contenedora */
85   .chat-container {
86     padding: 20px;
87     min-height: 100vh;
88     /* Fondo degradado para la página completa */
89     background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
90   }
91
92   /* Puedes añadir más estilos globales o específicos aquí,
93      o importarlos desde tu archivo .scss */
94 </style>
95
```

IMAGEN PENDIENTE

Descripción: Captura de pantalla del resultado final del "Ejemplo Completo".

Formato: PNG.

Contenido: Mostrar la aplicación de chat completamente estilizada, con el fondo degradado de la página, la `TConversacion` centrada, y los slots de cabecera (video) y entrada (input simple) aplicados.

7. Paso 6: Solución de Problemas (Troubleshooting)

7.1. Error: "Cannot read properties of undefined (reading 'fnEnviarMensaje')"

- **Causa:** La prop `mensajeClient` es `undefined`, `null`, o no es un objeto que implemente la interfaz `TMensajeClientInterface`.
- **Solución:** Asegurarse de que el objeto pasado a `mensajeClient` esté correctamente instanciado y contenga todas las funciones requeridas por la interfaz antes de renderizar `TConversacion`.

7.2. Error de CORS (Cross-Origin Resource Sharing)

- **Causa:** El servidor ISS (Azure Functions) no ha sido configurado para aceptar peticiones desde el origen del frontend (ej. `http://localhost:4321` o `https://mi-app.com`).
- **Solución:** En la configuración de la Azure Function App (servidor ISS), navegar a la sección "CORS" y añadir el origen del frontend (ej. `http://localhost:4321`) a la lista de orígenes permitidos.

7.3. El chat se conecta, pero los mensajes en tiempo real de otros usuarios no llegan

- **Causa:** La conexión WebSocket está activa, pero el cliente no se ha unido al grupo (sala) correcto. Esto casi siempre es debido a un `idConversacion` incorrecto o diferente entre los usuarios.
- **Solución:** Asegurarse de que ambos usuarios (en diferentes navegadores) estén usando *exactamente* el mismo `idConversacion` (sensible a mayúsculas/minúsculas).

8. Paso 7: Checklist de Implementación

- [] Importar `TConversacion`, `TTDTipoConversacion` y la interfaz `TMensajeClientInterface`.
- [] Crear o inyectar una instancia de un cliente que implemente `TMensajeClientInterface`.
- [] Definir `iusuarioActual` e `idConversacion`. El consumidor define **cómo** se establecen estos valores.
- [] Implementar el componente `TConversacion` en el HTML pasando todas las props requeridas: `mensajeClient`, `iusuarioActual`, `idConversacion`, y `ttdconversacion`.
- [] (Opcional) Personalizar con `styleHistorial` o `slot="cabecera"` para alinear con la identidad de tu app.